

if

consteval

Document #: P1938R0
Date: 2019-10-10
Project: Programming Language C++
EWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>
Richard Smith
<richard@metafoo.co.uk>
Andrew Sutton
<asutton@lock3software.com>
Daveed Vandevoorde
<daveed@edg.com>

Contents

- [1 Introduction](#)
- [2 Problems with Status Quo](#)
- [3 Proposal](#)
- [4 History](#)
- [5 Wording](#)
- [6 Acknowledgments](#)
- [7 References](#)

§ 1 Introduction

Despite this paper missing both our respective NB comment deadlines and the mailing deadline, we still believe it provides a significant enough improvement to the status quo that it should be considered for C++20.

C++20 will have several new features to aid programmers in writing code during constant evaluation. Two of these are `std::is_constant_evaluated` [P0595R2] and `consteval` [P1073R3], both adopted in San Diego 2018. `consteval` is for functions that can only be invoked during constant evaluation. `is_constant_evaluated` is a magic library function to check if the current evaluation is constant evaluation to provide, for instance, a valid implementation of an algorithm for constant evaluation time and a better implementation for runtime.

However, despite being adopted at the same meeting, these features interact poorly with each other and have other issues that make them ripe for confusion.

§ 2 Problems with Status Quo

There are two problems this paper wishes to address.

The first problem is the interplay between this magic library function and the new `constexpr`. Consider the example:

```
constexpr int f(int i) { return i; }

constexpr int g(int i) {
    if (std::is_constant_evaluated()) {
        return f(i) + 1; // <==
    } else {
        return 42;
    }
}

constexpr int h(int i) {
    return f(i) + 1;
}
```

The function `h` here is basically a lifted, constant-evaluation-only version of the function `g`. At constant evaluation time, they do the same thing, except that during runtime, you cannot call `h`, and `g` has this extra path. Maybe this code started with just `h` and someone decided a runtime version would also be useful and turned it into `g`.

Unfortunately, `h` is well-formed while `g` is ill-formed. You cannot make that call to `f` (that is ominously marked with an arrow) in that location. Even though that call will *only* happen during constant evaluation, that's still not enough.

With specific terms, the call to `f` `(i)` inside of `g` `( )` is an *immediate invocation* and needs to be a constant expression and it is not. Whereas the call to `f` `(i)` inside of `h` `( )` is *not* considered an *immediate invocation* because it is in an *immediate function context* (i.e. it's invoked from another immediate function), so it has a weaker set of restrictions that it needs to follow.

In other words, this kind of construction of conditionally invoking a `constexpr` function from a `constexpr` function just Does Not Work (modulo the really trivial cases - one could call `f(42)` for instance, just never `f(i)`).

We find this lack of composability of features to be problematic and think it can be improved.

The second problem is specific to `is_constant_evaluated`. Once you learn what this magic function is for, the obvious usage of it is:

```
size_t strlen(char const* s) {
    if constexpr (std::is_constant_evaluated()) {
        for (const char *p = s; ; ++p) {
            if (*p == '\0') {
                return static_cast<std::size_t>(p - s);
            }
        }
    } else {
        __asm__("SSE 4.2 insanity");
    }
}
```

```
}  
}
```

This example, inspired by [P1045R0], has a bug: it uses `if constexpr` to check the conditional `is_constant_evaluated` `()` rather than a simple `if`. You have to really deeply understand a lot about how constant evaluation works in C++ to understand that this is in fact not only *not* “obviously correct” but is in fact “obviously incorrect,” for some definition of obvious. This is such a likely source of error that Barry submitted bugs to both [gcc](#) and [clang](#) to encourage the compilers to warn on such improper usage. gcc 10.1 will provide a warning for the [simple case](#):

```
<source>: In function 'constexpr int f(int)':  
<source>:4:45: warning: 'std::is_constant_evaluated' always evaluates to true  
in 'if constexpr' [-Wtautological-compare]  
    4 |     if constexpr (std::is_constant_evaluated()) {  
      |                               ~~~~~^~
```

But then people have to understand why this is a warning, and what this even means. Nevertheless, a compiler warning is substantially better than silently wrong code, but it is problematic to have an API in which many users are drawn to a usage that is tautologically incorrect.

§ 3 Proposal

We propose a new form of `if` statement which is spelled:

```
if consteval { }
```

The braces (in both the `if` and the optional `else`) are mandatory and there is no condition. If evaluation of this statement occurs during constant evaluation, the first substatement is executed. Otherwise, the second substatement (if there is one) is executed.

This behaves exactly as today’s:

```
if (std::is_constant_evaluated()) { }
```

except with three differences:

1. No header include is necessary.
2. The syntax is different, which completely sidesteps the confusion over the proper way to check if we’re in constant evaluation. You simply cannot misuse the syntax.
3. We can use `if consteval` to allow invoking immediate functions.

To explain the last point a bit more, the current language rules allow you to invoke a `consteval` function from inside of another `consteval` function ([\[expr.const\]/12](#)) - we can do this by construction:

An expression or conversion is in an *immediate function context* if it is potentially evaluated and its innermost non-block scope is a function parameter scope of an immediate function.
An expression or conversion is an *immediate invocation* if it is an explicit or implicit

invocation of an immediate function and is not in an immediate function context. An immediate invocation shall be a constant expression.

By extending the term *immediate function context* to also include an `if` `constexpr` block, we can allow the second example to work:

```
constexpr int f(int i) { return i; }

constexpr int g(int i) {
    if constexpr {
        return f(i) + 1; // ok: immediate function context
    } else {
        return 42;
    }
}

constexpr int h(int i) {
    return f(i) + 1; // ok: immediate function context
}
```

Additionally, such a feature would allow for an easy implementation of the original `std::is_constant_evaluated()`:

```
constexpr bool is_constant_evaluated() {
    if constexpr {
        return true;
    } else {
        return false;
    }
}
```

Which in itself suggests that this is the more fundamental feature. As such, `std::is_constant_evaluated()` may itself no longer be necessary. However, given the very late date of this proposal, we would more than happily keep it if it allows us this new language feature.

§ 4 History

The initial revision of the `std::is_constant_evaluated()` proposal [[P0595R0](#)] was actually targeted as a language feature rather than a library feature. The original spelling was `if constexpr()`. The paper was presented in Kona 2017 and was received very favorably in the form it was presented (17-4). The poll to consider a magic library alternative was only marginally more preferred (17-3). We believe that in the two years since these polls were taken, having a dedicated language feature with an impossible-to-misuse API, that can coexist with the rest of the constant ecosystem, is the right direction.

§ 5 Wording

Extend the definition of immediate function context in 7.7 [expr.const] (and use bullet points):

An expression or conversion is in an *immediate function context* if it is potentially evaluated and either:

- (12.1) — its innermost non-block scope is a function parameter scope of an immediate function-
or
- (12.2) — it appears in the first *compound-statement* of an `if` `constexpr` statement ([stmt.if]).

Change 8.5 [stmt.select] to add the new grammar:

```
selection-statement:  
    if constexpropt ( init-statementopt condition ) statement  
    if constexpropt ( init-statementopt condition ) statement else statement  
+   if constexpr compound-statement  
+   if constexpr compound-statement else compound-statement  
    switch ( init-statementopt condition ) statement
```

Add a new clause to 8.5.1 [stmt.if]

If the `if` statement is of the form `if` `constexpr` and evaluation occurs in a context that is manifestly constant-evaluated ([expr.const]), the first substatement is executed. Otherwise, if the `else` part of the selection statement is present, then the second substatement is executed.

Change 20.15.10 [meta.const.eval] to use this new functionality:

```
constexpr bool is_constant_evaluated() noexcept;
```

- 1 ~~Returns: `true` if and only if evaluation of the call occurs within the evaluation of an expression or conversion that is manifestly constant-evaluated ([expr.const]).~~

- 1 Effects: Equivalent to:

```
if constexpr {  
    return true;  
} else {  
    return false;  
}
```

§ 6 Acknowledgments

Thank you to David Stone and Tim Song for working through these examples.

§ 7 References

[P0595R0] Daveed Vandevoorde. 2017. The “constexpr” Operator.

<https://wg21.link/p0595r0>

[P0595R2] Richard Smith, Andrew Sutton, Daveed Vandevoorde. 2018.

`std::is_constant_evaluated.`

<https://wg21.link/p0595r2>

[P1045R0] David Stone. 2018. constexpr Function Parameters.

<https://wg21.link/p1045r0>

[P1073R3] Richard Smith, Andrew Sutton, Daveed Vandevoorde. 2018. Immediate functions.

<https://wg21.link/p1073r3>